Audit de sécurité et de qualité — MANGUIFI

Audit en lecture seule · Branche master · 21 octobre 2026

> **Synthèse.** Le point le plus grave est la connexion employé par matricule seul : elle ouvre une session sans mot de passe ni code. Deux secrets de repli codés en dur permettent de forger des sessions ADMIN si les variables d'environnement manquent. Aucun fichier du projet n'a été modifié.

1. Vérifications exécutées

Contrôle	Résultat
`tsc --noEmit`	OK, aucune erreur
ESLint (src, prisma, scripts)	0 erreur, 5 warnings (variables inutilisées)
`npm audit --omit=dev`	18 vulnérabilités : 3 critiques, 10 élevées, 4 modérées, 1 faible
`next build`	Non lancé

2. Critique

CRITIQUE Connexion employé par matricule seul

Fichier : `src/app/actions/matricule-auth.ts` (`matriculeLoginAction`)

- Le matricule suit un format séquentiel (`MGF-2026-000123`). Une fois connu, il suffit à ouvrir une session employé, sans mot de passe ni code.
- La session donne accès à `/espace` : pointage, export RGPD complet (téléphone, email, date de naissance, numéro de pièce, photo) et dépôt de justificatifs.
- Le rate limit est indexé sur `x-forwarded-for`, un en-tête que le client peut falsifier : il peut donc être contourné.
- **Correctif proposé :** réutiliser le flux OTP email existant (`email-auth.ts`), ou exiger un PIN.

CRITIQUE Secrets de repli codés en dur

- **`src/lib/auth.ts`** : si `AUTH_SECRET` est absent, la clé de signature JWT retombe sur une chaîne publique. N'importe qui peut alors forger une session ADMIN.
- **`src/lib/crypto.ts`** : même repli pour `ENCRYPTION_KEY`. La clé HMAC des emails (`hashEmail`) devient alors publique.
- **Correctif proposé :** lever une erreur au démarrage en production si la variable manque.

3. Élevé

ÉLEVÉ Sessions non révoquées

- `getCurrentUser` ne vérifie pas `employee.status`. Un employé désactivé (INACTIF) garde l'accès jusqu'à 30 jours.
- La réinitialisation du mot de passe n'invalide pas les sessions existantes. La déconnexion ne fait que supprimer le cookie.
- **Correctif proposé :** contrôler le statut à chaque requête et ajouter un compteur de version de session sur `User`.

ÉLEVÉ Rate limiting inefficace

- `src/lib/rate-limit.ts` stocke les compteurs en mémoire, par instance : sans effet sur Vercel (serverless).
- La clé repose sur `x-forwarded-for` : contournable. Si l'en-tête est absent, tous les clients partagent le seau `"local"`.
- **Correctif proposé :** IP issue du proxy de confiance, et store partagé (Redis ou Postgres).

ÉLEVÉ Dépendances vulnérables

- `next` 16.3.1 : critique (plage 16.0.0 à 16.3.7), corrigé en 16.4.2.
- `@capacitor/android` et `@capacitor/ios` 8.5.0 : critique, corrigé en 8.5.3.
- `prisma` 6.x : élevé, correctif en 7.10.1. Mise à jour majeure, rupture possible.
- `axios`, `undici`, `follow-redirects`, `fast-uri`, `brace-expansion`, `sharp`, `source-map-js` : élevé, corrigeables par `npm audit fix` sans rupture.
- Next 16 a des changements d'API (voir AGENTS.md) : tester le build après la montée de version.

4. Moyen

MOYEN OTP de l'ancien chemin WhatsApp

- `src/lib/otp.ts` (utilisé par `whatsapp-auth.ts`) stocke le code en clair, sans compteur de tentatives. Seul le rate limit (contournable) limite les essais sur un code à 6 chiffres.
- Le chemin email (`email-otp.ts`) fait mieux : hash et 5 tentatives maximum. Il faudrait aligner `otp.ts` dessus.
- À vérifier : ce chemin est-il encore exposé dans l'interface ?

MOYEN CSP trop permissive

- `next.config.ts` : `script-src 'unsafe-inline'` affaiblit la protection contre le XSS. Le commentaire du fichier explique le compromis, mais la dette reste.

MOYEN Injection HTML dans les emails

- `src/lib/email.ts` (vers les lignes 158 à 170) insère `firstName`, `orgName` et `to` sans échappement. Un nom d'organisation malveillant peut produire du HTML dans un email envoyé depuis l'expéditeur officiel.

MOYEN Secret des tâches cron

- `src/app/api/cron/backup/route.ts` et `retention/route.ts` acceptent le secret en paramètre d'URL (`?secret=`), donc journalisé.
- La comparaison n'est pas en temps constant (`!==`).

5. Faible et qualité

- **Énumération de comptes :** messages distincts à l'inscription (« Un compte existe déjà »), à la liaison de téléphone et à la connexion OTP admin.
- `decryptDataUrl` (`src/lib/crypto.ts`) renvoie le texte chiffré en cas d'échec au lieu de lever une erreur.
- **Données binaires en base :** photos et justificatifs stockés en data URL dans Postgres, ce qui alourdit les lignes et les sauvegardes.
- **Warnings ESLint :** `decryptDataUrl` (company.ts), `now` (email-auth.ts), `hashPhone` (matricule-auth.ts), `ip` (whatsapp-auth.ts), `endOfDay` (dashboard.ts).
- **CI :** lint et build seulement, pas de tests (le script `test:concurrency` n'est pas lancé).

6. Points positifs

- Les 36 actions serveur exportées vérifient une session ou un rôle (contrôle heuristique, pas une preuve formelle).
- Les contrôles de périmètre (`assertEmployeeInScope` et équivalents) protègent contre l'IDOR.
- Mots de passe en bcrypt, tokens de réinitialisation stockés hachés, chiffrement AES-256-GCM.
- Réinitialisation sans énumération en production ; cron fermé par défaut si `CRON_SECRET` est absent.
- En-têtes de sécurité : HSTS, `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy`.
- `.env` jamais commité (vérifié dans l'historique git).

7. Ordre de traitement recommandé

1. Points 1 (matricule) et 2 (secrets de repli) avant toute mise en production.
2. Points 3 et 4 (sessions, rate limiting).
3. `npm audit fix` pour le point 5, puis montées de version de Next, Capacitor et Prisma.